

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions.

Assessment Tool Weightage: 40%

Questions: 4 Total Marks: 40

Dr.V.Parthipan

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

CSA07 — COMPUTER NETWORKS

CO3 Assessment Tool 2 — Case Study: Answers

CO3: Analyze the performance of core network protocols — TCP, UDP, HTTP, and DNS — under varying conditions

Case Study 1. Cloud Video Conferencing — Voice Distortion and Frozen Frames

Symptoms during peak hours: voice distortion, frozen video frames, delayed communication. Monitoring shows 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization.

a) *Is the Problem Primarily the Transport Protocol or the Application Layer?*

This is primarily a **transport-layer problem**, with application-layer buffering as a significant secondary contributor. Real-time audio/video should travel over UDP-based RTP, not TCP. The specific combination of symptoms reported here — frozen frames and delayed communication rather than a clean drop in quality — is the signature of TCP's reliability machinery being active on a media stream: every lost packet triggers a retransmission and the receiver has to wait for in-order delivery before it can play anything further, which is exactly what "freezing" and "delay" look like from a user's chair. If UDP is already in use, then the raw 8% loss and especially the 180 ms of jitter are hitting the stream directly with no retransmission-related delay, but they still need to be absorbed by the application's own jitter buffer and loss-concealment logic — and 180 ms of jitter is roughly 6–9× a typical VoIP jitter budget of 20–30 ms, so a normally-sized buffer will be overwhelmed regardless of the transport underneath it.

b) *Justification Based on Real-Time Multimedia Requirements*

Real-time multimedia has a fundamentally different priority from a file download or a web page: it needs **consistent low latency far more than perfect reliability**. A slightly degraded but on-time video frame is usable in a live call; a perfect frame that arrives 300 ms late is not — you can't rewind a live conversation. TCP is built around the opposite trade-off: its congestion control (slow start, additive-increase/multiplicative-decrease) deliberately slows the sending rate after loss, and its retransmission and in-order delivery guarantees mean a single lost segment blocks everything queued behind it (head-of-line blocking) until it is successfully re-sent. At 8% loss, a TCP-carried media stream would be almost continuously backing off and retransmitting, producing exactly the frozen-frame, delayed-audio pattern described. UDP has no such delay-inducing mechanism by design — it sends and moves on, deliberately leaving loss-handling to the application (skip the frame, apply forward error correction, or conceal it), which is the correct trade-off for anything live. This is precisely why RTP/RTCP over UDP, not TCP, is the industry-standard transport for video conferencing (WebRTC, SIP-based systems, etc.).

c) *Recommended Protocol-Level and Application-Level Improvements*

- Protocol level — confirm the actual media stream uses UDP-based RTP/RTCP (reserving TCP only for signalling, e.g. SIP or a WebRTC data channel), since carrying live media over TCP is the most likely root cause of the freezing/delay pattern.
- Protocol level — add Forward Error Correction (FEC) so a bounded amount of loss can be reconstructed on the receiving end without waiting for a retransmission, and use RTCP NACK-based selective retransmission only for high-value frames (e.g. video keyframes) rather than everything.

- Protocol level — use a real-time-aware congestion control scheme (e.g., Google Congestion Control, as used in WebRTC) that reacts to loss and jitter by adjusting the encoder's bitrate rather than by stalling the stream.
- Application level — replace any fixed-size jitter buffer with an adaptive one that resizes based on measured network jitter, since a static small buffer cannot absorb 180 ms of variance without dropping packets outright.
- Application level — use loss-tolerant codecs (e.g. Opus for audio, which has built-in packet-loss concealment; a video codec with temporal scalability so lower-priority enhancement layers can be dropped instead of freezing the whole frame).
- Application/network level — apply QoS marking (DSCP) to conferencing traffic so it is queued ahead of routine bulk data during peak business hours, since "fluctuating bandwidth utilization" suggests the media stream is contending with other traffic on the same links; where feasible, route conferencing traffic over a dedicated SD-WAN path or a nearby media relay (SBC/TURN server) rather than the unpredictable public internet path.

Case Study 2. Global Web Services — 800 ms DNS Lookup Time

A multinational company's web homepage is slow to load; average DNS lookup time measures roughly 800 ms. The company wants a redesigned DNS infrastructure that keeps working through server failures.

a) Reasons Behind the High DNS Resolution Time

- A "cold" (uncached) resolution can involve several sequential hops — local resolver → root server → TLD server → the company's own authoritative name server — and every hop adds a full round trip, which adds up quickly if any of those servers are geographically far from the requesting user.
- If the company's authoritative DNS is centralized (one server or one region), users on other continents must reach across long physical distances for every lookup, and physical distance sets a hard latency floor no server-side tuning can remove.
- Short or poorly-tuned TTLs mean records expire from local/ISP resolver caches quickly, forcing repeated full resolutions instead of fast cached answers.
- No DNS pre-fetching means the browser/application only starts resolving a hostname the moment it is actually needed, so the full lookup cost sits directly in the critical path of page load instead of being hidden ahead of time.
- Additional likely contributors: DNSSEC validation overhead if not optimized, under-provisioned DNS server capacity, or resolvers not using EDNS Client Subnet to get routed to a genuinely nearby answer.

b) Proposed DNS Architecture (Anycast DNS + Pre-Fetching + TTL Optimization)

- Anycast DNS — deploy the authoritative name servers at multiple geographically distributed points-of-presence, all advertising the same IP address via BGP anycast, so each user's query is automatically routed by the network to the nearest instance rather than one fixed, possibly distant, server.
- TTL optimization (tiered, not uniformly short or long) — give stable records (main website, apex domain) longer TTLs (an hour or more) to maximize cache hits and minimize repeat lookups, while records that need fast failover (load-balanced endpoints) keep short TTLs (30–60 s) so the system can still react quickly when something goes down.
- DNS pre-fetching — use pre-fetch hints (e.g. <link rel="dns-prefetch">) so the client resolves hostnames for resources it is likely to need next (API endpoints, CDN assets, third-party embeds) in the background, hiding resolution latency behind other page-load work instead of blocking on it.

- Supporting measures — run at least two independent, geographically diverse authoritative name servers (ideally with two different DNS providers) so a single provider's outage cannot remove resolution entirely; enable EDNS Client Subnet so even public resolvers direct users to the nearest regional endpoint.

With anycast collapsing physical distance down to "nearest point-of-presence" (typically well under 30 ms on a well-distributed network), tiered TTLs eliminating most repeat full lookups, and pre-fetching hiding whatever latency remains from the user's perceived load time, bringing the average lookup below the 50 ms target is realistic.

c) Advantages and Limitations of the Proposed Architecture

- Performance — anycast, tiered TTLs, and pre-fetching each attack the 800 ms bottleneck from a different angle, so together they compound rather than just adding up. Limitation: anycast latency depends on BGP routing behaviour, which is not always perfectly aligned with true geographic proximity, and aggressive pre-fetching can waste some DNS query volume on resources that end up unused.
- Scalability — anycast scales naturally: adding more points-of-presence increases both capacity and coverage without any change required on the client side, and load spreads across many nodes instead of concentrating on one server as the user base grows. Limitation: operating many geographically distributed, zone-synchronized DNS nodes adds real operational cost and complexity.
- Availability — anycast gives automatic failover: if a node fails, BGP withdraws its route and traffic shifts to the next-nearest healthy node with no client-side reconfiguration; pairing this with two independent DNS providers protects against a provider-wide outage as well. Limitation: BGP route withdrawal is not instantaneous, so a brief disruption is possible for the affected region during failover, and any lag in zone-data synchronization across nodes can cause temporarily inconsistent answers.

Case Study 3. Global Cloud Provider — Centralized DNS Bottleneck

A cloud provider serving millions of users worldwide sees DNS queries taking nearly 800 ms because every request is directed to a single primary DNS server, which becomes especially painful during promotional-event traffic spikes.

a) Impact of Centralized DNS Architecture on Application Performance

- Single point of contention — every query worldwide funnels through one server, so as global traffic grows (and especially spikes during promotions), queueing delay stacks on top of pure network latency.
- Distance penalty — users far from that one server's physical location pay a propagation-delay cost that no amount of server-side capacity can remove; this is a physics limit (signal speed through fiber), not a tuning problem.
- Single point of failure — if the primary server goes down, is overloaded, or loses network reachability, resolution fails for every user everywhere, taking the service fully offline rather than just degrading it — a severe availability risk for a provider serving millions of users.
- No geographic load shedding — a centralized design cannot spread a traffic spike across locations, so a promotional event anywhere in the world adds directly to the same single queue, whereas a distributed design would absorb that spike locally.

b) Distributed DNS Solution Design

- Anycast-based authoritative DNS clusters deployed across major regions (e.g. North America, Europe, Asia-Pacific, South America), all sharing the same anycast IP addresses so queries are automatically routed to the nearest cluster.

- Layered caching — recursive resolvers at the ISP/edge level cache per-record TTLs, supplemented by regional caching resolvers close to users to shave off further round trips.
- Multi-provider redundancy — authoritative DNS split across at least two independent providers, so a full outage at one vendor does not remove resolution capability company-wide.
- Integration with Global Server Load Balancing (GSLB) — DNS answers are tailored to each query's origin so users are pointed not just to the nearest DNS node but to the nearest healthy application data center, combining name resolution with traffic engineering.
- Health-checked failover — automated health checks on backend origin servers so DNS/GSLB stops directing traffic toward an unhealthy or overloaded region during a spike, redirecting to the next-nearest healthy region instead.
- Proactive capacity planning — real-time DNS query monitoring so extra anycast capacity can be provisioned ahead of known, predictable spikes (like a scheduled promotional event) rather than reacting after the fact.

c) How TTL, Anycast Routing, and DNS Caching Influence Performance and Fault Tolerance

- TTL — shorter TTLs let the system react and fail over faster (clients re-resolve sooner after a change), but cost more frequent full lookups and slightly higher average load/latency; longer TTLs maximize caching efficiency and reduce load, but slow down how quickly clients notice a change like a failover. This is a direct, tunable trade-off between raw performance and responsiveness to failure — which is why tiered TTLs (as in Case Study 2) are the practical answer rather than one fixed value everywhere.
- Anycast routing — improves performance by sending each query to the topologically nearest instance, cutting round-trip time close to a physical minimum; improves fault tolerance because BGP automatically reroutes around a failed or withdrawn node with no client-side change needed. Its effectiveness does depend on healthy, well-maintained BGP peering — a misconfigured route can occasionally send traffic somewhere less optimal.
- DNS caching — cuts both perceived latency (a cached answer is served almost instantly, with no round trip at all) and load on the authoritative servers (fewer queries ever reach the origin, so the same infrastructure supports far more total clients); the trade-off is that a long-cached, now-stale answer can delay how quickly a client notices and routes around a failure — the same tension TTL governs, viewed from the caching side.
- Together — anycast answers "which server should I reach," caching answers "how often do I actually need to ask," and TTL is the dial that balances both of those against how fast the whole system can respond to a failure. A well-tuned distributed DNS design needs all three working together, not just one of them in isolation.

Case Study 4. Stock Trading Platform — Order Acknowledgment Latency Spike (1 ms → 40 ms)

During market opening, average order-acknowledgment latency rises from 1 ms to 40 ms. Engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

a) Three TCP-Related Factors Behind the Latency Increase

- Flow control — TCP throttles the sender based on the receiver's advertised window (how much buffer space it currently has free). Under the burst of orders at market open, if the receiving application or OS socket buffer cannot drain incoming data fast enough, the advertised window shrinks, and the sender is forced to pause or slow down — adding latency directly in the path of every order acknowledgment right when volume is highest.

- Nagle's algorithm — by default, TCP briefly buffers small outgoing segments so several small writes can be combined into fewer, larger packets, which helps bulk/interactive traffic but actively works against an application that sends many small, latency-critical messages like order acknowledgments. Its interaction with delayed ACKs on the other side is a well-known cause of tens-to-hundreds of milliseconds of added delay on exactly this kind of small-message traffic.
- SYN queue limitations — every new TCP connection starts with a SYN handshake, and the server's SYN backlog queue has a finite size. If the rate of new connection attempts during market open exceeds what the server can accept in time, additional SYNs queue up or get dropped (forcing client-side retransmission of the SYN itself), inflating connection establishment time — a serious issue if the platform opens a new connection per order rather than reusing persistent ones.

b) How Each Factor Affects Transaction Processing in High-Frequency Systems

- Flow control — produces send-side stalls that directly delay when an acknowledgment can actually leave the server, which lines up precisely with the observed rise from 1 ms to 40 ms; in a high-frequency trading context, even single-digit-millisecond delays can be the difference between an order filling at the intended price and it filling at a worse one (slippage), so this is a real financial impact, not just a UX inconvenience.
- Nagle's algorithm — adds artificial, protocol-level buffering delay to exactly the small, frequent acknowledgment messages a trading system depends on. The extra latency here is not the network being slow — it is the TCP stack intentionally holding data before transmitting it, which is especially damaging for round-trip-sensitive order confirmations.
- SYN queue limitations — delays the very first message of any new order/trading session, and if the platform pays this handshake cost per order (rather than reusing a connection), every order at peak volume risks queueing or a dropped SYN — producing the kind of large, inconsistent latency spikes that are worst exactly when consistent low latency matters most.

c) Recommended TCP Configuration Changes

- Disable Nagle's algorithm on the trading platform's sockets (the `TCP_NODELAY` option exists specifically for this situation) to remove the artificial buffering delay on small acknowledgment messages entirely.
- Tune TCP send/receive buffer sizes and enable window scaling so legitimate high-burst traffic is not unnecessarily throttled by flow control, and ensure the receiving application reads/drain its socket promptly so the advertised window does not collapse under load.
- Increase the SYN backlog (e.g. Linux `net.core.somaxconn` / `net.ipv4.tcp_max_syn_backlog`) to absorb the connection-attempt burst at market open, and enable SYN cookies so the server degrades gracefully under an even larger-than-expected spike rather than dropping legitimate attempts.
- Use persistent, reused (keep-alive/pooled) TCP connections instead of opening a new connection per order, which avoids paying the handshake cost — and any SYN queue risk — on every single transaction.
- Review delayed-ACK behaviour on the receiving side as well, since it can reintroduce small-message latency even after Nagle's algorithm is disabled on the sender.
- For the most latency-critical paths, consider kernel-bypass/user-space networking (e.g. DPDK-based NICs) or a custom low-latency protocol on UDP — an option real high-frequency trading firms use once standard, even well-tuned, TCP stack latency becomes the limiting factor.